

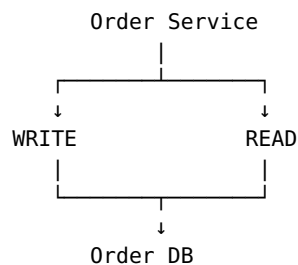
CQRS - Step by Step in Spring Boot

CQRS — Step by Step in Spring Boot

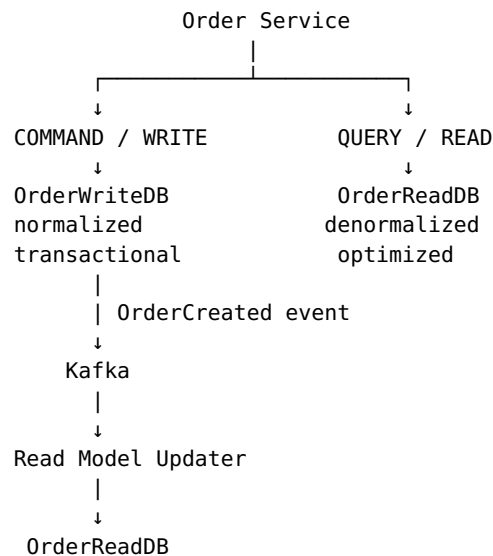
Learn **CQRS step by step by coding it in Spring Boot**, from the simple version to the production version.

1. First understand the idea

Without CQRS:



With CQRS:



The key point:

| **Command = change data Query = read data**

2. Create Spring Boot project

For learning, create:

cqrs-demo

Dependencies:

Spring Web

Spring Data JPA
MySQL Driver
Lombok

For the first implementation, **don't add Kafka**. We will first understand CQRS with one database.

3. Create Order Entity

```
@Entity
@Table(name = "orders")
@Getter
@Setter
@NoArgsConstructor
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String customerName;

    private Double amount;

    private String status;
}
```

Database:

```
orders
-----
id
customer_name
amount
status
```

This is our **write model**.

4. Create Write Repository

```
public interface OrderWriteRepository
    extends JpaRepository<Order, Long> {
}
```

This repository is responsible for:

```
INSERT
UPDATE
DELETE
```

5. Create Command DTO

A command represents an instruction to change something.

```
@Getter
@Setter
public class CreateOrderCommand {

    private String customerName;

    private Double amount;
}
```

For example:

```
{  
  "customerName": "Moni",  
  "amount": 1000  
}
```

This means:

| Create an order.

6. Create Command Service

```
@Service  
@RequiredArgsConstructor  
public class OrderCommandService {  
  
    private final OrderWriteRepository orderWriteRepository;  
  
    public Long createOrder(CreateOrderCommand command) {  
  
        Order order = new Order();  
  
        order.setCustomerName(command.getCustomerName());  
        order.setAmount(command.getAmount());  
        order.setStatus("CREATED");  
  
        Order savedOrder =  
            orderWriteRepository.save(order);  
  
        return savedOrder.getId();  
    }  
}
```

Notice:

```
Command  
↓  
Command Service  
↓  
Write Repository  
↓  
Database
```

7. Create Command Controller

```
@RestController  
@RequestMapping("/orders")  
@RequiredArgsConstructor  
public class OrderCommandController {  
  
    private final OrderCommandService orderCommandService;  
  
    @PostMapping  
    public ResponseEntity<Long> createOrder(  
        @RequestBody CreateOrderCommand command) {  
  
        Long orderId =  
            orderCommandService.createOrder(command);  
  
        return ResponseEntity.ok(orderId);  
    }  
}
```

Test:

POST /orders

Request:

```
{
  "customerName": "Moni",
  "amount": 1000
}
```

Response:

1

So far we have implemented the **Command side**.

8. Now create the Query side

We don't want our read API to directly expose the write entity.

Create a separate DTO:

```
@Getter
@Setter
@AllArgsConstructor
public class OrderSummaryDto {

    private Long id;

    private String customerName;

    private Double amount;

    private String status;
}
```

9. Create Query Repository

For the simple CQRS implementation, we can still use the same database.

```
public interface OrderReadRepository
    extends JpaRepository<Order, Long> {

    Optional<Order> findById(Long id);
}
```

Important:

Same DB
Different responsibility
Different repository
Different service

This is still CQRS.

CQRS **does not require two databases**.

10. Create Query Service

```
@Service
@RequiredArgsConstructor
public class OrderQueryService {

    private final OrderReadRepository orderReadRepository;
```

```

    public OrderSummaryDto getOrder(Long id) {

        Order order =
            orderReadRepository.findById(id)
                .orElseThrow(() ->
                    new RuntimeException(
                        "Order not found"));

        return new OrderSummaryDto(
            order.getId(),
            order.getCustomerName(),
            order.getAmount(),
            order.getStatus()
        );
    }
}

```

Flow:

```

GET /orders/1
  ↓
Query Controller
  ↓
Query Service
  ↓
Read Repository
  ↓
Database

```

11. Create Query Controller

```

@RestController
@RequestMapping("/orders")
@RequiredArgsConstructor
public class OrderQueryController {

    private final OrderQueryService orderQueryService;

    @GetMapping("/{id}")
    public ResponseEntity<OrderSummaryDto> getOrder(
        @PathVariable Long id) {

        return ResponseEntity.ok(
            orderQueryService.getOrder(id)
        );
    }
}

```

Test:

GET /orders/1

Response:

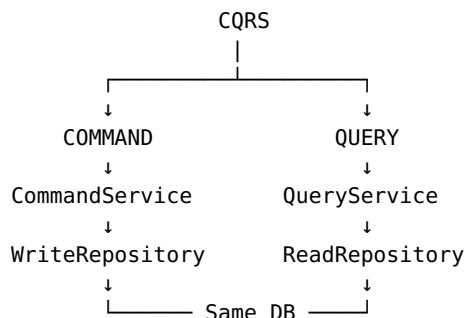
```

{
  "id": 1,
  "customerName": "Moni",
  "amount": 1000.0,
  "status": "CREATED"
}

```

12. What did we actually achieve?

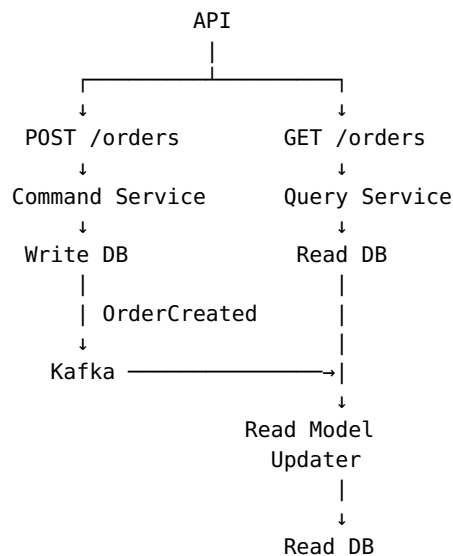
Our application now has:



This is **logical CQRS**.

13. Real CQRS — Separate databases

Now let's make it more realistic.



For example:

Write DB

MySQL:

```

orders
-----
id
customer_id
amount
status
created_at
  
```

Read DB

MongoDB / Elasticsearch:

```

{
  "orderId": 1001,
  "customerName": "Moni",
  "amount": 1000,
  "status": "CREATED",
  "productName": "iPhone",
  "merchantName": "ABC Store"
}
  
```

The read model can contain **pre-joined/denormalized data**.

Therefore:

```
100 DB joins
  ↓
Read model
  ↓
One fast lookup
```

14. Add Kafka

Now after creating an order:

```
OrderCommandService
  ↓
MySQL
  ↓
OrderCreatedEvent
  ↓
Kafka
  ↓
ReadModelUpdater
  ↓
MongoDB
```

Event:

```
public record OrderCreatedEvent(
    Long orderId,
    String customerName,
    Double amount,
    String status
) {
}
```

15. Publish event

```
@Service
@RequiredArgsConstructor
public class OrderCommandService {

    private final OrderWriteRepository orderWriteRepository;

    private final KafkaTemplate<String, OrderCreatedEvent>
        kafkaTemplate;

    public Long createOrder(CreateOrderCommand command) {

        Order order = new Order();

        order.setCustomerName(command.getCustomerName());
        order.setAmount(command.getAmount());
        order.setStatus("CREATED");

        Order saved =
            orderWriteRepository.save(order);

        OrderCreatedEvent event =
            new OrderCreatedEvent(
                saved.getId(),
                saved.getCustomerName(),
```

```

        saved.getAmount(),
        saved.getStatus()
    );

    kafkaTemplate.send(
        "order-created",
        event
    );

    return saved.getId();
}
}

```

16. Read Model Updater

Now another component listens to Kafka.

```

@Component
@RequiredArgsConstructor
public class OrderReadModelUpdater {

    private final OrderReadRepository orderReadRepository;

    @KafkaListener(
        topics = "order-created",
        groupId = "order-read-model"
    )
    public void handleOrderCreated(
        OrderCreatedEvent event) {

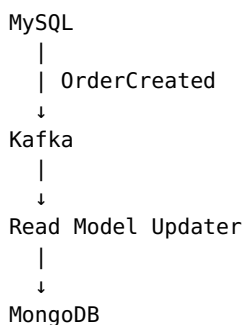
        OrderReadModel readModel =
            new OrderReadModel();

        readModel.setOrderId(event.orderId());
        readModel.setCustomerName(event.customerName());
        readModel.setAmount(event.amount());
        readModel.setStatus(event.status());

        orderReadRepository.save(readModel);
    }
}

```

Now:



17. Important concept — Eventual Consistency

Suppose:

```

10:00:00.000
POST /orders

```

↓

10:00:00.010
MySQL saved

↓

10:00:00.020
Kafka event

↓

10:00:00.050
MongoDB updated

There can be a small delay.

Therefore:

Write DB = latest
Read DB = eventually latest

This is called:

| **Eventual Consistency**

18. What if Kafka fails?

This is a very important interview question.

Bad implementation:

```
orderRepository.save(order);

kafkaTemplate.send(event);
```

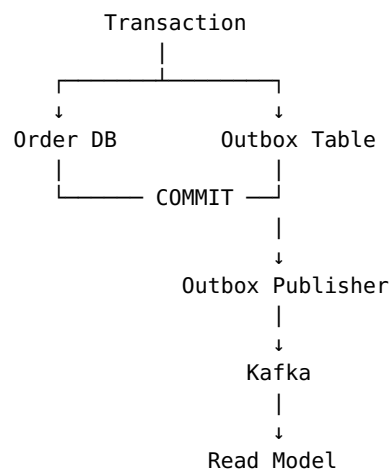
Problem:

MySQL SAVE
↓
SUCCESS

Kafka
↓
FAILED ✕

Now the order exists but the read model never receives the event.

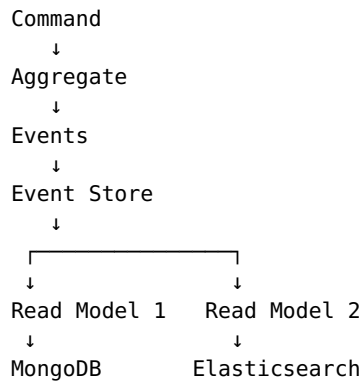
Production solution: Transactional Outbox



This is one of the most important **4-7 year experience** CQRS concepts.

19. CQRS + Event Sourcing

You can go one step further:



Example events:

OrderCreated
PaymentCompleted
OrderShipped
OrderDelivered

Instead of only storing:

Order Status = DELIVERED

you store the history:

OrderCreated
↓
PaymentCompleted
↓
OrderShipped
↓
OrderDelivered

Then you can rebuild the read model from events.

20. CQRS interview answer

For **5+ years Java/Spring Boot**, you can answer:

CQRS stands for Command Query Responsibility Segregation. It separates the write side, which handles commands and transactional state changes, from the read side, which handles queries and can use a denormalized model optimized for reading. CQRS doesn't necessarily require two databases; it can start as a logical separation within one database. In larger systems, the write and read models can use separate stores and Kafka can synchronize them asynchronously, resulting in eventual consistency. For reliable event publishing, I would use the Transactional Outbox pattern. CQRS is useful for read-heavy systems, complex reporting, or when read and write workloads have very different requirements, but it adds operational complexity and is unnecessary for simple CRUD applications.

Remember this:

CQRS

- Command → WRITE
- Query → READ
- Same DB possible
- Separate DB possible
- Kafka → synchronization
- Eventual consistency
- Outbox → reliable events
- Event Sourcing → optional

This is the **CQRS coding path** recommended for learning: **simple same-DB CQRS → Kafka read model → separate MySQL/MongoDB → Outbox → Event Sourcing.**